

Malla de Seguridad, Privacidad y Resiliencia

Ecosistema de Automatización IA Autónomo — Vector Consultores

Autor: Matías Medrano · Módulo 8, Proyecto Integrador · Orquestador: n8n · Base: appmVz5S3bfSSMfgU

1. El problema que este documento resuelve

Un sistema autónomo que interactúa con clientes reales tiene tres formas de causar daño, y ninguna requiere mala intención:

- Puede filtrar datos.** Cada llamada a una API externa es un dato que sale de la organización. Si el sistema envía el mensaje completo de un cliente a un modelo de lenguaje sin filtrar, está exportando información personal a un tercero sin necesidad operativa.
- Puede romperse en silencio.** Una automatización que falla y no avisa es peor que una que no existe: genera la ilusión de que el trabajo está hecho. Un lead que entró un viernes y murió en un nodo caído no reaparece el lunes; simplemente se perdió.
- Puede afirmar cosas falsas con total seguridad.** Un modelo de lenguaje no distingue entre lo que sabe y lo que completa. Si redacta una propuesta comercial y le falta el dato del precio, la tendencia natural del modelo es rellenar el hueco con algo plausible. Ese texto, enviado sin revisión, es un compromiso comercial que la empresa no asumió.

Este documento describe las tres capas que se implementaron contra esos riesgos: **minimización de datos, rutas de contingencia y semáforos de validación humana**. La lógica de fondo es que ninguna de las tres capas alcanza sola. La minimización reduce lo que se puede filtrar, los error handlers evitan la falla silenciosa, y el control humano es lo único que frena una alucinación antes de que salga al mundo.

2. Política de minimización de datos

El principio operativo es simple: **el sistema procesa lo mínimo necesario para hacer su trabajo y descarta el resto lo antes posible**. Cada dato que no se recolecta es un dato que no se puede filtrar, perder ni exponer.

2.1 Qué se descarta en la puerta de entrada

Un email entrante trae mucho más que su contenido: cabeceras técnicas, IP de origen, píxeles de seguimiento, firmas con datos de contacto, cadenas completas de respuestas anteriores. El nodo `FN_Normalizar_Payload` actúa como filtro de entrada y conserva **seis campos**, descartando todo lo demás antes de que toque cualquier otro componente.

Se conserva	Por qué es necesario
<code>message_id</code>	Control anti-duplicado
<code>thread_id</code>	Responder dentro de la conversación
<code>from.email</code> y <code>from.domain</code>	Identificar la cuenta y poder responder
<code>subject</code>	Contexto de clasificación
<code>body_clean</code>	El contenido a interpretar
<code>attachments</code> (metadatos)	Decidir si hay lectura densa

Se descarta	Riesgo que se elimina
Cabeceras SMTP e IP de origen	Datos de geolocalización sin uso operativo
Píxeles y enlaces de seguimiento	Exposición a terceros de la actividad interna
Firmas de correo	Cargos, teléfonos directos, domicilios que nadie pidió
Cadena de respuestas previas	Conversaciones con terceros ajenos al lead
HTML, estilos y adjuntos de imagen	Superficie de ataque y volumen inútil

El descarte ocurre **antes** de cualquier llamada a la IA. Esto no es solo higiene de privacidad: el nodo de validación `IF_Validar_CamposMinimos` corta el flujo si faltan datos, con lo cual un mensaje incompleto nunca genera un gasto de tokens.

2.2 Truncado del cuerpo

`body_clean` está limitado a 8.000 caracteres por esquema. El límite cumple dos funciones a la vez: acota el volumen de información personal que sale hacia el proveedor de IA, y evita que un email con veinte respuestas encadenadas dispare el costo de tokens sin aportar contexto útil.

2.3 Qué recibe efectivamente el modelo de IA

Este es el punto donde la mayoría de las automatizaciones filtran datos sin darse cuenta: se le pasa al modelo el objeto entero “por si acaso”. Acá la selección es explícita.

Nodo	Recibe	recibe
AI_Clasificar_Lead_Mini	asunto, cuerpo, tipo de cliente	teléfono, email completo, historial de montos
AI_Analizar_Claude_VIP	cuerpo, texto del adjunto, historial de servicios	datos de facturación, contactos de otras cuentas

El nombre del contacto se conserva porque la respuesta debe ser personalizada. El teléfono no viaja a ningún modelo: solo lo usa el nodo de WhatsApp, ya en la etapa de envío, y solo si el envío fue aprobado.

2.4 Anonimización en los registros de error

Cuando algo falla, la reacción instintiva es guardar todo el contexto para poder depurar. Es también la forma más común de que datos personales terminen replicados en una tabla de logs que nadie revisa y que se comparte con más gente que la base principal.

`LOG_Errores` guarda el payload **enmascarado**, y el enmascarado ocurre antes de escribir, no después:

```
Original:  l.gimenez@empresa.com      → Guardado:  l.g***@empresa.com
Original:  +5493411234567             → Guardado:  +549341***4567
Original:  "Buenos días, somos una..." → Guardado:  [cuerpo omitido · 1.240 car.]
```

Se conserva el dominio porque es lo que permite diagnosticar si un error afecta a una cuenta específica. Se pierde la identidad individual, que para depurar no hace falta.

2.5 Retención

Dato	Retención	Fundamento
Cuerpo_Original	90 días, luego purga automática	Después de tres meses el texto ya no tiene valor operativo
Clasificación, score, tiempos	Indefinida	Es la serie histórica que alimenta los KPIs; no contiene datos personales
LOG_Errores	180 días	Ventana suficiente para análisis de patrones de falla
INT_Interacciones	Indefinida	Es la memoria del cliente, su razón de ser

La distinción importante: se purga el **contenido**, no el **registro**. El sistema conserva que hubo un lead VIP de tal empresa, con tal score, respondido en tantos minutos. Deja de conservar lo que la persona escribió textualmente.

2.6 Credenciales y superficie de exposición

Regla	Implementación
Cero claves en nodos	Todas las credenciales viven en el gestor de n8n, referenciadas por nombre
Cero claves en el repositorio	El <code>.json</code> exportado se sanitiza antes de subir: se verifica que no queden tokens, IDs de base ni URLs de webhook privadas
Cero claves en evidencias	Screenshots y video se revisan uno por uno antes de publicar
Vista pública sin datos personales	El dashboard oculta <code>Email_Contacto</code> , <code>Telefono</code> y <code>Cuerpo_Original</code>
Acceso de solo lectura	La Shared View no permite edición ni descarga de la base completa

Sobre el **Base ID**: se documenta abiertamente porque no es un secreto. Funciona como el número de una casa — identifica, pero no abre. Lo que abre es la API key, y esa nunca sale del gestor de credenciales. Confundir ambas cosas lleva a dos errores opuestos: ocultar identificadores inofensivos y, por costumbre, terminar tratando con la misma ligereza a las claves que sí importan.

3. Rutas de contingencia (Error Handlers)

3.1 Arquitectura del manejo de errores

n8n permite designar un **Error Workflow** global: un flujo separado que se dispara automáticamente cuando cualquier nodo del flujo principal falla sin ser capturado. Esa es la red de seguridad de último recurso.

Pero un sistema que depende solo de la red de último recurso es frágil. El diseño usa **tres niveles**:

Nivel 1 — Prevención. Nodos de validación que cortan antes de que el error ocurra. `IF_Validar_CamposMinimos` verifica que haya email válido y cuerpo con contenido; `IF_AntiLoop` verifica que el mensaje no se haya procesado antes. Un mensaje que no pasa estos filtros no genera error: genera un registro de descarte, que es distinto.

Nivel 2 — Reintento con degradación. Los nodos de red tienen `Retry On Fail` con dos reintentos y espera creciente. La mayoría de los fallos de API son transitorios; reintentar resuelve sin intervención. El nodo `FN_Validar_JSON_IA` implementa un caso especial: si el modelo devuelve JSON inválido, reintenta una vez con `temperature: 0`, que reduce la variabilidad y suele producir salida bien formada.

Nivel 3 — Captura y registro. Si el reintento falla, el Error Workflow captura el fallo, lo clasifica, lo registra en `LOG_Errores` y decide si alerta.

3.2 Matriz de escenarios de falla

Escenario	Cómo se detecta	Ruta de contingencia	Estado final	¿Alerta?
API de IA caída o timeout	Error del nodo <code>AI_</code>	Retry x2 con espera creciente → Error Workflow → log + alerta	Error	Sí, si persiste
La IA devuelve JSON malformado	Falla de validación en <code>FN_Validar_JSON_IA</code>	1 reintento con <code>temperature: 0</code> → si falla, se deriva a revisión humana con el texto crudo	En revisión humana	No
Datos faltantes en el mensaje	<code>IF_Validar_CamposMinimos</code>	Rama false → registro de descarte. No se llama a la IA	Descartado	No
Credencial vencida o revocada	Error <code>AUTH_FAIL</code>	Escalamiento inmediato, sin reintentos: reintentar con credencial inválida solo consume operaciones	Error	Sí, inmediata
Bucle: el sistema se responde a sí mismo	<code>IF_AntiLoop</code> detecta remitente propio	Corte inmediato del flujo	Descartado	No
Mensaje duplicado	<code>Message_ID</code> ya existe en la base	Corte antes de crear registro	—	No
Exceso de ejecuciones por hora	Contador contra <code>MAX_EJECUCIONES_HORA</code>	Pausa del trigger + alerta	—	Sí
El humano no responde	Timeout de 24 h en <code>WAIT_Aprobacion_HITL</code>	Recordatorio en Slack; a las 48 h escala al supervisor. No se envía nada	En revisión humana	Sí, a las 48 h
Airtable no disponible	Error del nodo <code>AT_</code>	Retry x3 → si persiste, el payload queda en la ejecución para reanudar manualmente	Error	Sí
Adjunto ilegible o corrupto	Falla de extracción de texto	Continúa sin el adjunto y lo marca en el borrador para que el humano lo abra	En revisión humana	No

3.3 El Error Workflow paso a paso

Nodo	Función
ERR_Trigger_Global	Error Trigger de n8n, designado como Error Workflow del flujo principal
FN_Clasificar_Error	Categoriza en cinco tipos: <code>API_FAIL</code> , <code>DATA_MISSING</code> , <code>PARSE_FAIL</code> , <code>AUTH_FAIL</code> , <code>TIMEOUT</code>
AT_Log_Error	Escribe en <code>LOG_Errores</code> con nodo, mensaje, severidad y payload anonimizado
IF_Es_Critico	Evalúa si es <code>AUTH_FAIL</code> o si hubo tres fallos en la última hora
SLK_Alerta_Ops	Alerta al canal de operaciones, solo para casos críticos
AT_Update_Estado_Error	Marca el lead como <code>Error</code> para que quede visible en el dashboard

La clasificación en cinco tipos convierte el error de anécdota en métrica. Si sube `API_FAIL` , el problema es del proveedor y no hay nada que corregir del lado propio. Si sube `PARSE_FAIL` , el prompt se degradó y hay que revisar la versión. Si sube `DATA_MISSING` , cambió algo en el formato de entrada. Sin esa taxonomía, todos los errores se ven iguales y ninguno se puede accionar.

3.4 Por qué la alerta es selectiva

Todos los errores se registran; solo los críticos alertan. Un canal de Slack que notifica cada timeout transitorio se vuelve ruido en una semana, y el ruido se ignora — incluido el mensaje importante que llegue después. La alerta se reserva para lo que requiere acción humana inmediata: credenciales caídas, fallos repetidos y aprobaciones vencidas.

3.5 Los tres controles del check de seguridad

Control exigido	Implementación
Filtro anti-bucle infinito	Triple barrera: <code>IF_AntiLoop</code> descarta mensajes cuyo remitente sea una dirección del propio sistema; verifica que el <code>Message_ID</code> no exista ya en la base; y un contador corta si se superan las ejecuciones por hora definidas en <code>MAX_EJECUCIONES_HORA</code>
Comparación de tipos correctos	<code>Score_IA</code> es de tipo Number en Airtable y se castea con <code>parseInt()</code> antes de cualquier comparación. El umbral se lee de <code>CFG_Config</code> y también se convierte a número. La comparación es número contra número, nunca texto contra texto
Prompt dinámico con variables del sistema	Los prompts se leen de <code>PRM_Prompts</code> y se inyectan variables con notación <code>{{ }}</code> . Ningún nodo tiene texto de prompt escrito literalmente. La versión usada se persiste en cada ejecución

Sobre el segundo control vale una aclaración, porque es el que más silenciosamente rompe sistemas: si `Score_IA` fuera texto, la comparación se haría carácter por carácter y `"9"` resultaría mayor que `"85"` . El router mandaría leads al segmento equivocado **sin arrojar ningún error**. Nada en los logs indicaría el problema. Es el peor tipo de falla, porque solo aparece cuando alguien audita resultados y nota que los VIP no son los que deberían.

4. Semáforos de validación humana (Human-in-the-loop)

4.1 El principio de diseño

El sistema es **autónomo en el procesamiento y supervisado en la interacción con el exterior**. Todo lo que ocurre puertas adentro — clasificar, buscar contexto, redactar, registrar — sucede sin intervención. Todo lo que sale hacia una persona real requiere una firma humana.

Esta división no es una concesión a la desconfianza en la IA: es una asignación de responsabilidad. Un modelo de lenguaje no puede asumir un compromiso comercial en nombre de la empresa. Una persona sí. El punto HITL es donde esa responsabilidad se transfiere explícitamente y queda registrada.

El curso lo llama **efecto metralla**: el escenario en que una automatización mal calibrada dispara mil mensajes erróneos antes de que alguien lo note. El HITL es el seguro contra ese escenario, y por eso está ubicado exactamente en el último punto reversible del flujo.

4.2 Los cuatro semáforos

#	Punto	Ubicación	Riesgo que mitiga	Quién decide
HITL-1	Antes del envío al cliente	Entre <code>AT_Guardar_Borrador</code> y <code>GML_Enviar_Respuesta</code>	Alucinación en precios o plazos, tono inadecuado, respuesta a un mensaje mal clasificado	Responsable comercial, en Slack
HITL-2	Baja confianza del modelo	Rama alternativa cuando <code>confianza < 0,60</code> o <code>requiere_intervencion_humana = true</code>	El modelo mismo señala que no está seguro; ignorar esa señal sería desperdiciarla	Responsable comercial
HITL-3	Antes de WhatsApp proactivo	Previo a <code>WSP_Alerta_VIP</code>	Riesgo regulatorio: un mensaje proactivo a un número personal tiene un marco normativo más estricto que un email de respuesta	Responsable comercial
HITL-4	Revisión del informe mensual	Sobre la salida del sub-flujo de Batches	Deriva del criterio de clasificación a lo largo del tiempo	Supervisor

4.3 Por qué HITL-1 está exactamente ahí

La ubicación de un punto de control humano define su utilidad. Demasiado temprano, y el humano revisa datos crudos que no le dicen nada. Demasiado tarde, y revisa algo que ya salió.

`HITL-1` está en **el último punto reversible**: el borrador ya existe, con toda la información necesaria para evaluarlo, pero todavía no salió nada hacia el cliente. Un paso antes, el revisor vería el email original sin el análisis; un paso después, estaría leyendo lo que el cliente ya recibió.

4.4 Diseño de la interfaz de aprobación

Un botón de “Aprobar” sin contexto convierte al humano en un sello de goma. Si aprobar es más fácil que leer, todo se aprueba, y el control existe solo en el papel. El mensaje de Slack está diseñado contra ese fallo:

- **Muestra el score y la confianza** de la clasificación, para que el revisor sepa cuánta atención merece ese caso puntual.
- **Muestra qué modelo generó el texto**, porque un borrador de Claude sobre un pliego denso merece más lectura que una respuesta estándar breve.
- **Ofrece tres salidas, no dos**. Aprobar, **Editar** y Rechazar. La opción de editar es la que evita el falso dilema: sin ella, un borrador casi correcto obliga a elegir entre aprobar algo imperfecto o rechazar y rehacer todo a mano. La mayoría de la gente elige aprobar.
- **Registra quién decidió**. El campo `Aprobado_Por` se persiste en la base. Ante un reclamo, queda establecido quién autorizó el envío y cuándo.

4.5 Qué pasa si nadie responde

El estado por defecto ante la inacción es **no enviar**. A las 24 horas el sistema envía un recordatorio; a las 48 escala al supervisor. En ningún momento hay un envío automático por vencimiento de plazo.

Es una decisión deliberada y tiene un costo: un lead puede quedar sin respuesta si el equipo está desbordado. Se acepta ese costo porque el daño de una demora es recuperable y el de un envío equivocado no. El backlog de aprobaciones pendientes es, además, un KPI visible del dashboard, justamente para que la demora no pase inadvertida.

4.6 El ciclo de mejora

Cuando el revisor rechaza un borrador, el motivo se registra en `Motivo_Rechazo`. Esos motivos son la materia prima para ajustar el prompt: si tres rechazos seguidos señalan que el tono es demasiado formal, el prompt se corrige y la versión nueva se guarda en `PRM_Prompts` con número propio.

Como cada ejecución persiste su `Prompt_Version`, se puede correlacionar una caída de la tasa de aprobación con el cambio exacto que la causó. El control humano deja de ser solo un freno y pasa a ser la fuente de datos que mejora al sistema.

5. Cómo se verifica que esto funciona

Las tres capas se probaron explícitamente durante el test de estrés. No alcanza con que el camino feliz funcione: hay que provocar las fallas.

Prueba	Qué se provoca	Qué debe ocurrir	Evidencia
Datos faltantes	Mensaje con cuerpo vacío	Corta en la validación, no llama a la IA, registra el descarte	03-camino-infeliz-datos-faltantes.png
API caída	Credencial de IA inválida a propósito	Reintenta, deriva al Error Workflow, registra y alerta	04-error-handler-disparado.png
Rechazo humano	El aprobador presiona Rechazar	No se envía nada; se guarda el motivo	05-slack-aprobacion.png
Duplicado	Se reinyecta el mismo Message_ID	Se detecta y no se reprocesa	08-log-errores.png
Baja confianza	Mensaje deliberadamente ambiguo	Se deriva a HITL-2 en lugar de decidir solo	07-airtable-registros.png

La prueba del rechazo humano es la más importante de las cinco, porque valida la afirmación central de este documento: **que el sistema efectivamente no puede contactar a un cliente sin autorización**. Una arquitectura que dice tener un punto de control pero nunca lo probó bajo rechazo no tiene un control, tiene una intención.

6. Síntesis

Capa	Contra qué protege	Mecanismo principal
Minimización de datos	Filtración innecesaria a terceros	Seis campos conservados de todo lo que entra; anonimización en logs; purga a 90 días
Rutas de contingencia	Falla silenciosa y pérdida de trabajo	Tres niveles: prevención, reintento, captura global con taxonomía de cinco tipos
Validación humana	Alucinación y compromiso comercial no autorizado	Cuatro semáforos, con el principal en el último punto reversible del flujo

Ninguna de las tres capas es suficiente por sí sola, y el orden importa: la minimización reduce la superficie del problema, los error handlers evitan que el sistema falle sin avisar, y el control humano es lo único que se interpone entre una afirmación inventada por un modelo y un cliente real leyéndola.